

CS 2614- Computer Organization

Computer Organization - the way in which hardware components are connected together to create a computer system.

Computer Architecture - structure and behavior of individual functional components of the computer.

Computer Design - development of computer hardware system given a set of specifications.

Chapter 1

Logic Gates

Boolean Algebra

Map Simplification

Combinational Circuits

Flip-Flops

Sequential Circuits

BINARY SIGNALS

MA-4

LOGIC GATES

- AND
- OR
- NOT
- NOR
- NAND

MA-8

BOOLEAN ALGEBRA

- Logic diagram
- Logic diagram $\rightarrow$ Boolean function
- Boolean function $\rightarrow$ Logic Diagram
- Truth table
- Truth table $\rightarrow$ Function
- Function $\rightarrow$ Truth table

MA-20

LOGIC SIMPLIFICATION

- Boolean identities and laws
- Logic simplification using Boolean Algebra
- Principle of duality
- DeMorgan's theorem
- NAND gate implementation

MA-29

LOGIC SIMPLIFICATION USING KARNAUGH MAP

- Mapping functions to Karnaugh map
- Karnaugh map rules
- Don't care condition
- Product of sums from Karnaugh map

MA-37

COMBINATIONAL CIRCUIT

- Half adder
- Full adder
- BCD to Excess-3 converter

MA-57

SEQUENTIAL CIRCUIT

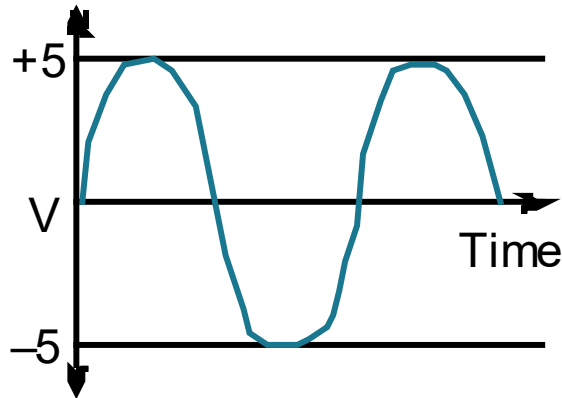
- Flip Flops
- Analysis of sequential circuits
- State table
- Flip Flop excitation tables
- Design of sequential circuit

MA-64

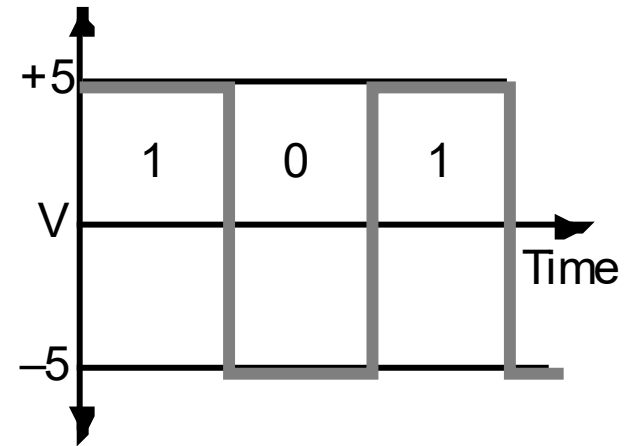
BINARY SIGNALS

Digital Systems

Digital vs. Analog Waveforms



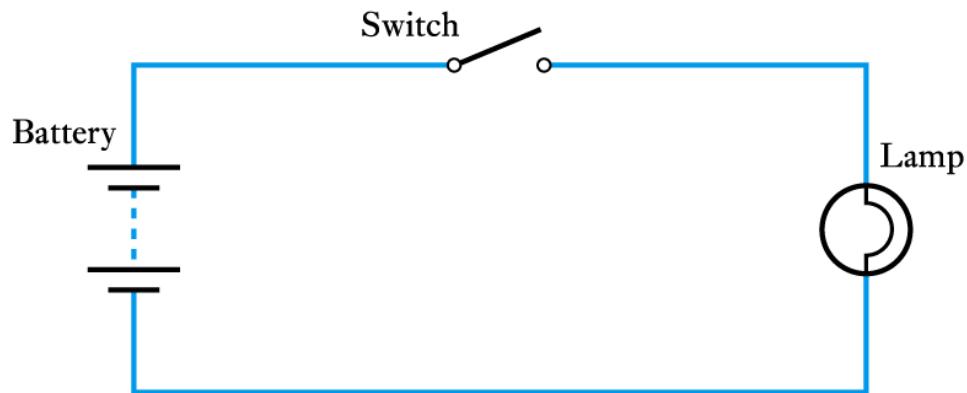
Analog:
Values vary over a broad range
over time



Digital:
Only assumes discrete values

Binary Signals

- Digital systems consists of mostly binary signals
- A **binary quantity** is one that can take only 2 states



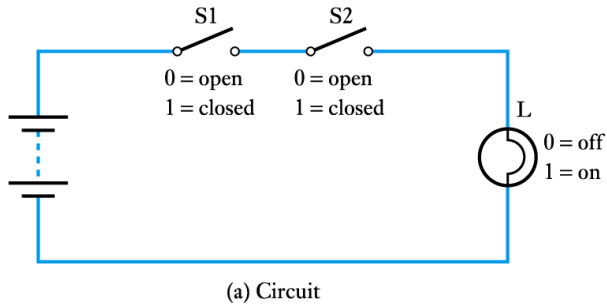
A simple binary arrangement

<i>S</i>	<i>L</i>
OPEN	OFF
CLOSED	ON

<i>S</i>	<i>L</i>
0	0
1	1

A truth table

- A binary arrangement with two switches in series

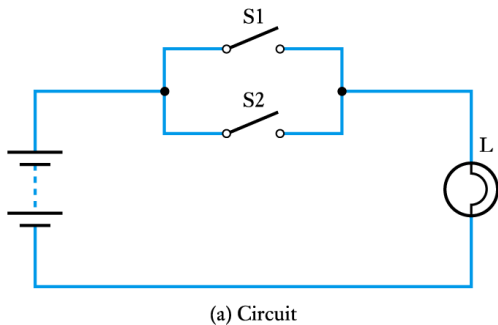


S1	S2	L
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

$$L = S1 \text{ AND } S2$$

- A binary arrangement with two switches in parallel

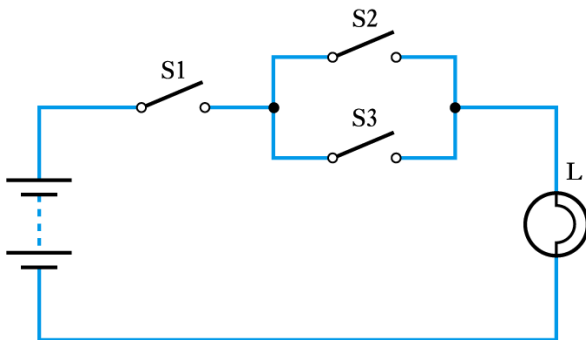


S1	S2	L
0	0	0
0	1	1
1	0	1
1	1	1

(b) Truth table

$$L = S1 \text{ OR } S2$$

- A series/parallel arrangement



S1	S2	S3	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$L = S1 \text{ AND } (S2 \text{ OR } S3)$$

LOGIC GATES

- AND
- OR
- NOT
- NOR
- NAND

Logic Gates

- The building blocks used to create digital circuits are **logic gates**
- There are three elementary logic gates and a range of other simple gates
- Each gate has its own **logic symbol** which allows complex functions to be represented by a logic diagram
- The function of each gate can be represented by a **truth table** or using **Boolean notation**

The AND gate



(a) Circuit symbol

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

$$C = A \cdot B$$

(c) Boolean expression

The OR gate



(a) Circuit symbol

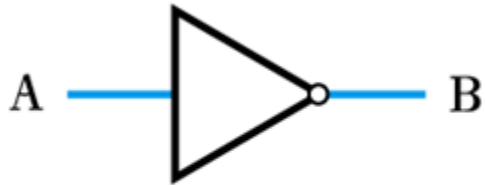
A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

(b) Truth table

$$C = A + B$$

(c) Boolean expression

The NOT gate (or inverter)



(a) Circuit symbol

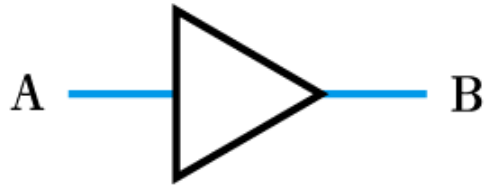
A	B
0	1
1	0

(b) Truth table

$$B = A'$$

(c) Boolean expression

A logic buffer gate



(a) Circuit symbol

A	B
0	0
1	1

(b) Truth table

$$B = A$$

(c) Boolean expression

The NAND gate



(a) Circuit symbol

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

(b) Truth table

$$C = (A \cdot B)'$$

(c) Boolean expression

The NOR gate



(a) Circuit symbol

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

(b) Truth table

$$C = (A + B)'$$

(c) Boolean expression

The Exclusive OR gate (or Difference)



(a) Circuit symbol

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table

$$C = A \oplus B$$

(c) Boolean expression

The Exclusive NOR gate (or Equivalence)



(a) Circuit symbol

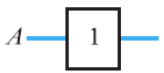
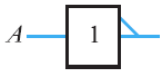
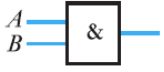
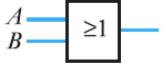
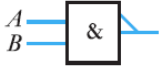
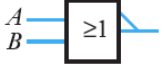
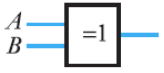
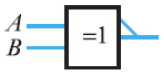
A	B	C
0	0	1
0	1	0
1	0	0
1	1	1

(b) Truth table

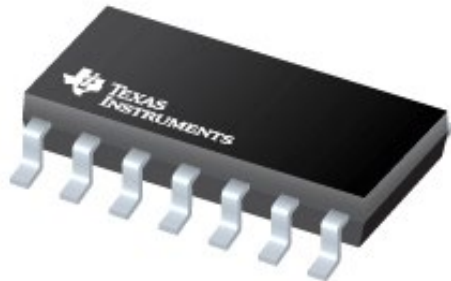
$$C = (A \oplus B)'$$

(c) Boolean expression

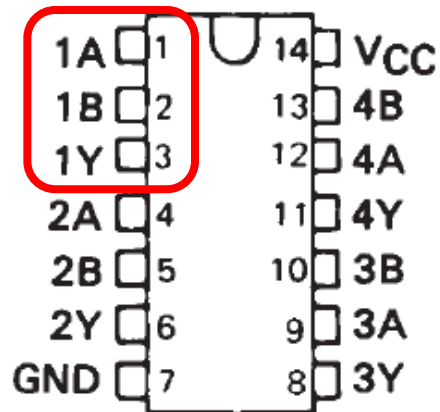
Summary of gates

Function	Symbol	Alternative symbol	Boolean expression	Truth table															
Buffer			$B = A$	<table><tr><th>A</th><th>B</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	B	0	0	1	1									
A	B																		
0	0																		
1	1																		
NOT			$B = \overline{A}$	<table><tr><th>A</th><th>B</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	B	0	1	1	0									
A	B																		
0	1																		
1	0																		
AND			$C = A \cdot B$	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	C	0	0	0	0	1	0	1	0	0	1	1	1
A	B	C																	
0	0	0																	
0	1	0																	
1	0	0																	
1	1	1																	
OR			$C = A + B$	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	C	0	0	0	0	1	1	1	0	1	1	1	1
A	B	C																	
0	0	0																	
0	1	1																	
1	0	1																	
1	1	1																	
NAND			$C = \overline{A \cdot B}$	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	C	0	0	1	0	1	1	1	0	1	1	1	0
A	B	C																	
0	0	1																	
0	1	1																	
1	0	1																	
1	1	0																	
NOR			$C = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	C	0	0	1	0	1	0	1	0	0	1	1	0
A	B	C																	
0	0	1																	
0	1	0																	
1	0	0																	
1	1	0																	
Exclusive OR			$C = A \oplus B$	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	C	0	0	0	0	1	1	1	0	1	1	1	0
A	B	C																	
0	0	0																	
0	1	1																	
1	0	1																	
1	1	0																	
Exclusive NOR			$C = \overline{A \oplus B}$	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	C	0	0	1	0	1	0	1	0	0	1	1	1
A	B	C																	
0	0	1																	
0	1	0																	
1	0	0																	
1	1	1																	

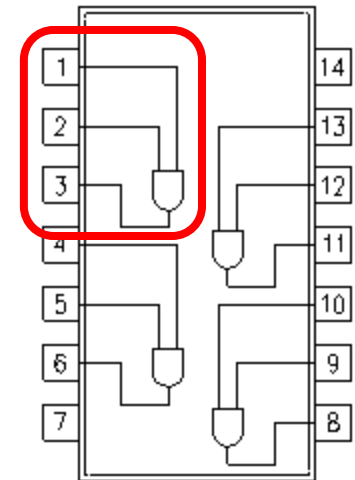
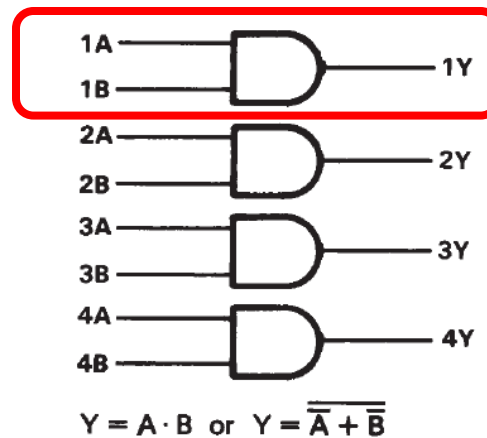
Data Sheet for Logic Gates



(TOP VIEW)



logic diagram (positive logic)



7408
Quad 2-Input
NOR Gate

BOOLEAN ALGEBRA

- Logic diagram
- Logic diagram $\rightarrow$ Boolean function
- Boolean function $\rightarrow$ Logic Diagram
- Truth table
- Truth table $\rightarrow$ Function
- Function $\rightarrow$ Truth table

Boolean Algebra

- **Boolean Algebra**
 - Deals with **binary variables** and **logic operations**
- **Boolean/Binary Variables**
 - variables that can only take the values '0' or '1'
 - Example: x, y, y', A, B
- **Boolean Functions**
 - A function expressed in terms of **binary variables** and **logic operations**. Output of the function is 0 or 1.
 - Ex: $F = x + y'z$

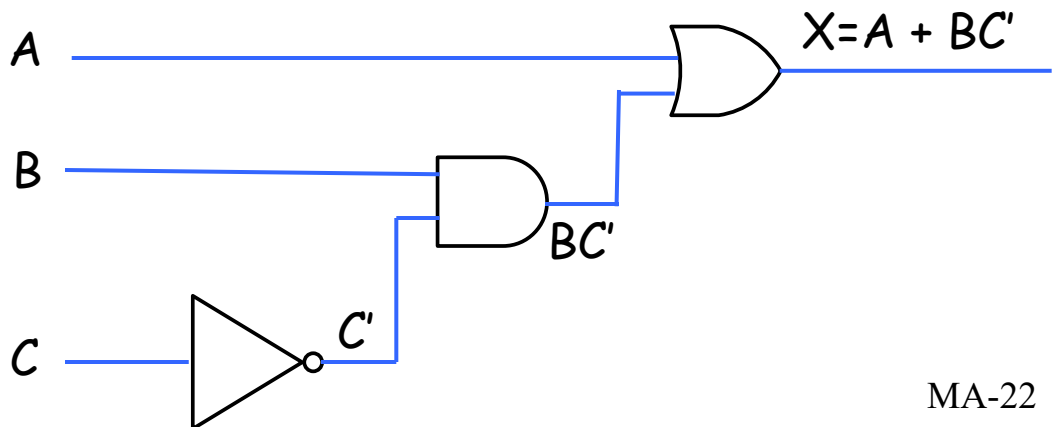
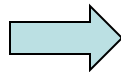
Logic Diagram

- A Boolean function can be represented as a Logic Diagram

Order of **precedence**: Expressions within parenthesis are evaluated first using the following order

- (1) Parenthesis,
- (2) NOT,
- (3) AND,
- (4) OR

$$X = A + BC'$$



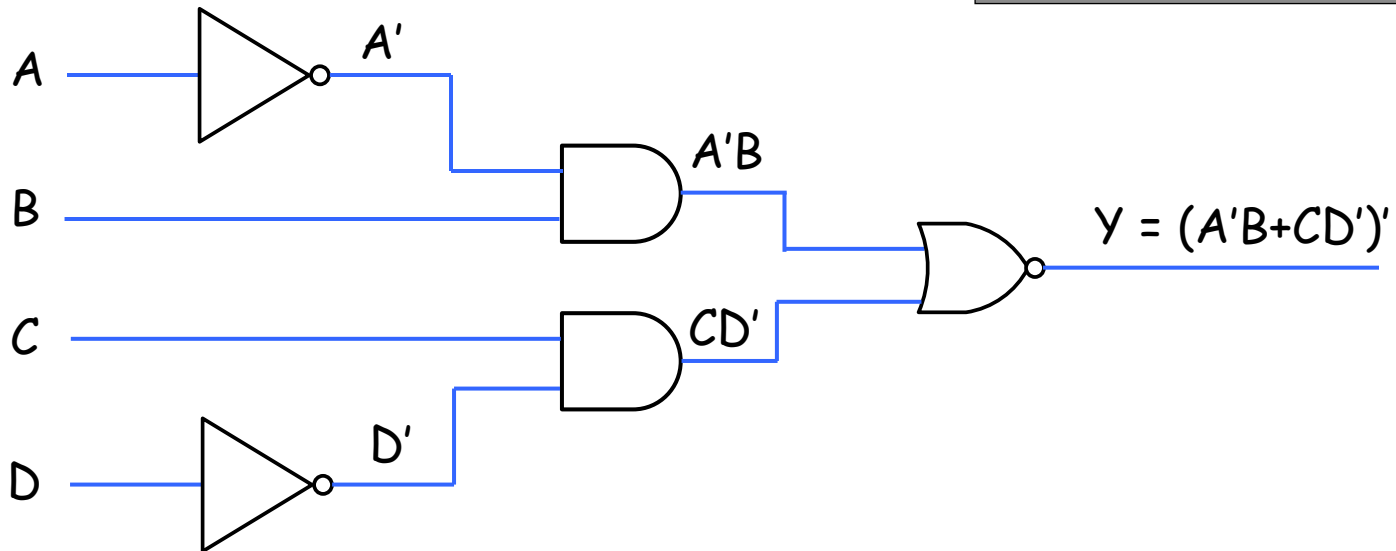
Implement a logic diagram from a Boolean function

- Example: Implement the function

$$Y = (A'B + CD')'$$

Order of **precedence**:

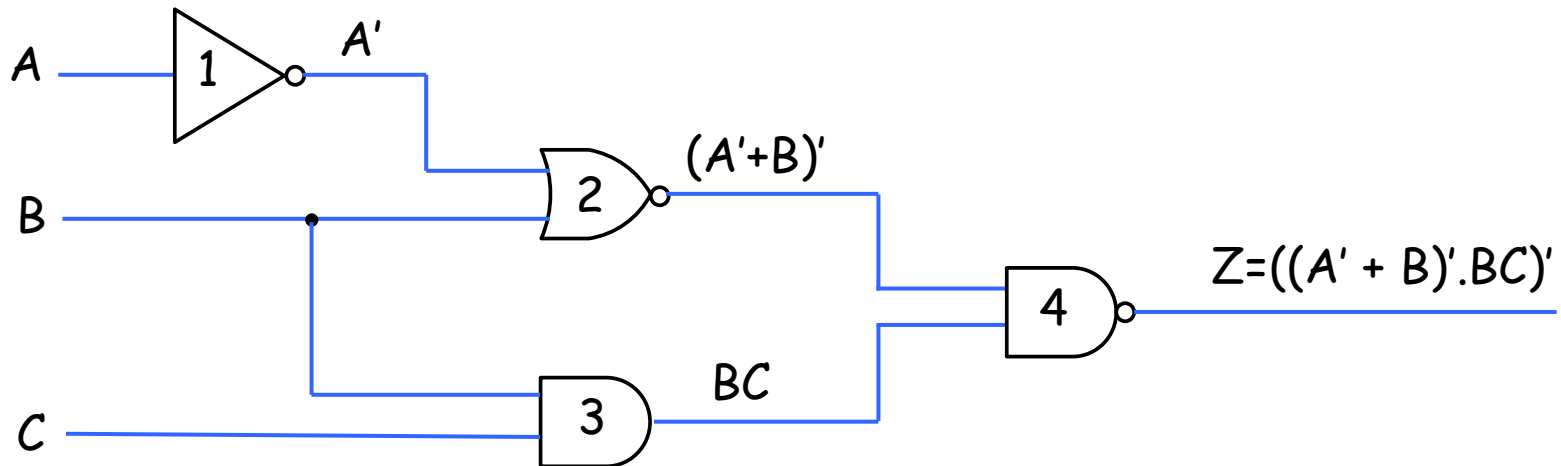
- (1) Parenthesis,
- (2) NOT,
- (3) AND,
- (4) OR



Generate a function from logic diagram

- Example: Generate the Boolean function for Z.

Work progressively from the **inputs to the output** adding logic expressions to the output of each gate in turn



Truth Table

A truth table is a breakdown of a logic function by listing all possible values the function can attain

- Example: Mr. and Ms. Boren have two children, John and Jane. When eating out they will go to a restaurant that serves only hamburgers or one that serves only chicken. Before going out, the family votes to decide on the restaurant. The majority wins, except when Mom and Dad agree, and in that case, they win. Any other tie votes produce a trip to the chicken restaurant.

Chicken 0
Hamburger 1

A	B	C	D	F
Dad	Mom	John	Jane	Vote
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Truth Table

Truth tables and functions

Minterms/Maxterms

Each combination of the variables in a truth table in the form of a product term is called a *minterm*.

x	y	z	Minterm	Maxterm
0	0	0	$x'y'z' = m_0$	$x+y+z = M_0$
0	0	1	$x'y'z = m_1$	$x+y+z' = M_1$
0	1	0	$x'yz' = m_2$	$x+y'+z = M_2$
0	1	1	$x'yz = m_3$	$x+y'+z' = M_3$
1	0	0	$xy'z' = m_4$	$x'+y+z = M_4$
1	0	1	$xy'z = m_5$	$x'+y+z' = M_5$
1	1	0	$xyz' = m_6$	$x'+y'+z = M_6$
1	1	1	$xyz = m_7$	$x'+y'+z' = M_7$

Note that $m_j = M_j'$

Specifying functions

- Functions can be expressed using minterms or maxterms.

Sum of Products

$$X = f(A,B,C) = \sum m(1,5,6)$$

$$= \sum (1,5,6)$$

$$= A'B'C + AB'C + ABC'$$

Product of Sums

$$X = f(A,B,C) = \prod M(0,2,3,4,7)$$

$$= \prod (0,2,3,4,7)$$

$$= (A+B+C) (A+B'+C) (A+B'+C')$$

$$(A'+B+C) (A'+B'+C')$$

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Truth table $\rightarrow$ function

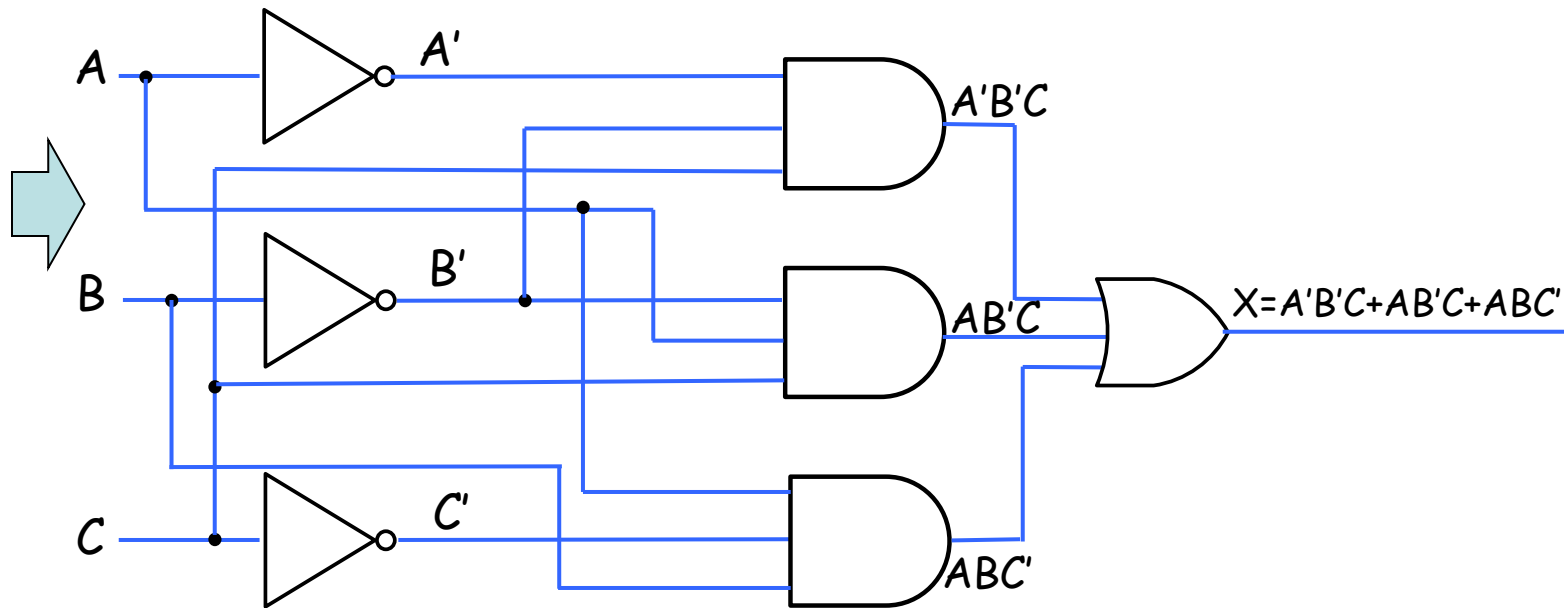
- Example: Implement the function of the following truth table using Sum-of-Products

1. First write down a Boolean expression for the output

$$X = A'B'C + AB'C + ABC'$$

2. Then draw the logic diagram

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



Function → truth table

- Example: Draw the truth table for $L = DX' + A$

Insert values of variables D, X and A in the logic function $L = DX' + A$ to obtain the truth table

□ TABLE 2-5

Truth Table for the Function $L = D\bar{X} + A$

D	X	A	L
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

$$L = D.X' + A$$
$$0.1' + 1 = 1$$

LOGIC SIMPLIFICATION

- Boolean identities and laws
- Logic simplification using Boolean Algebra
- Principle of duality
- DeMorgan's theorem
- NAND gate implementation

Logic Simplification

In some cases it is possible to *simplify* logic expressions using the rules of Boolean algebra

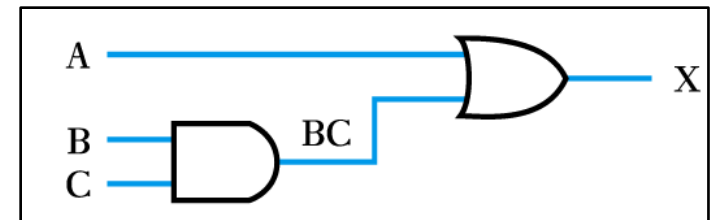
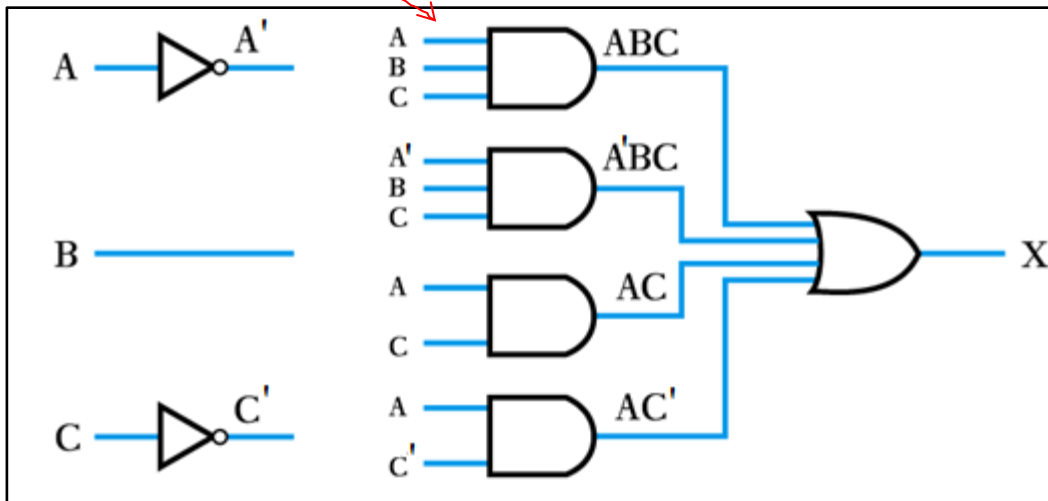
Example:

$$X = ABC + A'BC + AC + AC'$$

can be simplified to

$$X = BC + A$$

Hence the following circuits are equivalent



Boolean Identities and Laws

Identities

AND Function	OR Function	NOT function
$A \bullet 0 = 0$	$A + 0 = A$	$0' = 1$
$A \bullet 1 = A$	$A + 1 = 1$	$1' = 0$
$A \bullet A = A$	$A + A = A$	$(A')' = A$
$A \bullet A' = 0$	$A + A' = 1$	

Laws

Commutative law $AB = BA$ $A + B = B + A$	Absorption law $A + AB = A$ $A(A + B) = A$
Distributive law $A(B + C) = AB + AC$ $A + BC = (A + B)(A + C)$	De Morgan's law $(A + B)' = A' \cdot B'$ $(A \cdot B)' = A' + B'$
Associative law $A(BC) = (AB)C$ $A + (B + C) = (A + B) + C$	

Logic Simplification: Examples

Example: Simplify the expression

$$F = (A + B)(A + C)$$

$$\begin{aligned}
 F &= (A + B).(A + C) \\
 &= A.A + A.C + A.B + B.C \\
 &= A + A.C + A.B + B.C \\
 &= A.1 + A.C + A.B + B.C \\
 &= A(1 + C) + A.B + B.C \\
 &= A.1 + A.B + B.C \\
 &= A(1 + B) + B.C \\
 &= A.1 + B.C \\
 &= A + (B.C)
 \end{aligned}$$

- Distributive law
- AND identity ($A.A = A$)
- AND identity ($A.1 = A$)
- Distributive law
- OR Identity ($A + 1 = 1$)
- Distributive law
- OR Identity ($A + 1 = 1$)
- AND Identity ($A.1 = A$)

Example: Simplify $F = x'yz + x'yz' + xz$

$$\begin{aligned}
 F &= x'yz + x'yz' + xz \\
 &= x'y(z+z') + xz \\
 &= x'y.1 + xz \\
 &= x'y + xz
 \end{aligned}$$

Boolean Identities and Laws

Identities	AND Function	OR Function	NOT function
	$A \bullet 0 = 0$	$A + 0 = A$	$0' = 1$
	$A \bullet 1 = A$	$A + 1 = 1$	$1' = 0$
	$A \bullet A = A$	$A + A = A$	$(A')' = A$
	$A \bullet A' = 0$	$A + A' = 1$	
Laws	Commutative law $AB = BA$ $A + B = B + A$		Absorption law $A + AB = A$ $A(A + B) = A$
	Distributive law $A(B + C) = AB + AC$ $A + BC = (A + B)(A + C)$		De Morgan's law $(A + B)' = A'.B'$ $(A.B)' = A' + B'$
	Associative law $A(BC) = (AB)C$ $A + (B + C) = (A + B) + C$		

MA-31

Example:

Prove $x'y'z' + x'yz' + xyz'$
 $= x'z' + yz'$

Proof:

$$\begin{aligned}
 x'y'z' + x'yz' + xyz' &= x'y'z' + x'yz' + x'yz' + xyz' \\
 &= x'z'(y' + y) + yz'(x' + x) \\
 &= x'z'.1 + yz'.1 \\
 &= x'z' + yz'
 \end{aligned}$$

Principle of Duality

- ° Principle of *duality*: If an expression is valid in Boolean algebra, the dual of that expression is also valid.
- ° To form the dual of an expression,

Replace	with
+ (OR)	. (AND)
.	+
1	0
0	1

° *Take care not to alter the location of the parentheses if they are present.*

- Example: Form the dual of the expression
 $a + (bc) = (a + b)(a + c)$
- Answer: Following the replacement rules...
 $a(b + c) = ab + ac$

De Morgan's Theorem

A key theorem in simplifying Boolean algebra expression is DeMorgan's Theorem. It states:

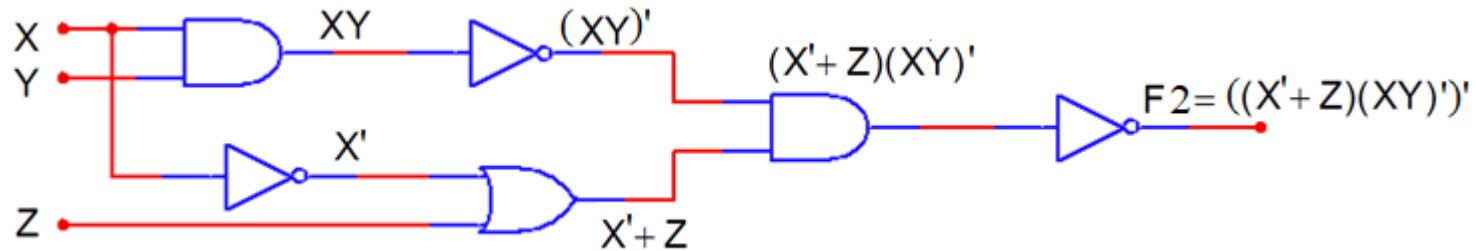
$$(a + b)' = a'b'$$

$$(ab)' = a' + b'$$

Example 1: Complement the expression $F = a(b + z(x + a'))$ and simplify.

$$\begin{aligned} F' &= (a(b + z(x + a')))' \\ &= a' + (b + z(x + a'))' \\ &= a' + b'(z(x + a'))' \\ &= a' + b'(z' + (x + a'))' \\ &= a' + b'(z' + x'a) \end{aligned}$$

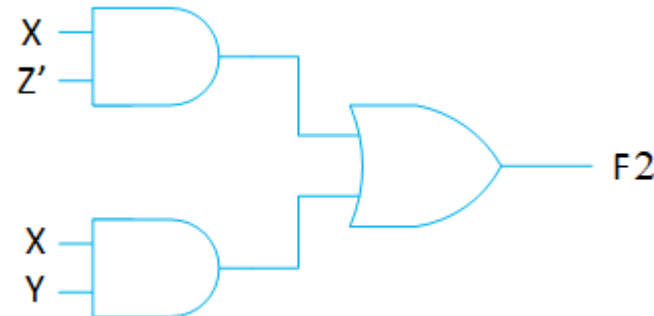
Example 2: Simplify the output function F2 using De Morgan's theorem



Much
Simplified

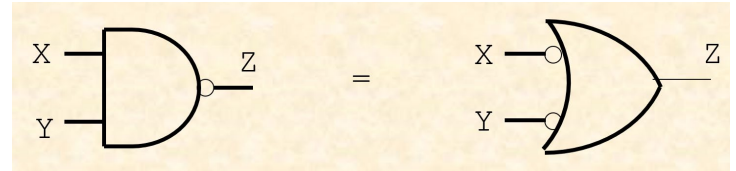
Solution:

$$\begin{aligned}
 F2 &= ((X' + Z)(XY)')' \\
 &= (X' + Z)' + ((XY)')' \\
 &= (X' + Z)' + (XY) \\
 &= (X')'Z' + (XY) \\
 &= XZ' + XY
 \end{aligned}$$



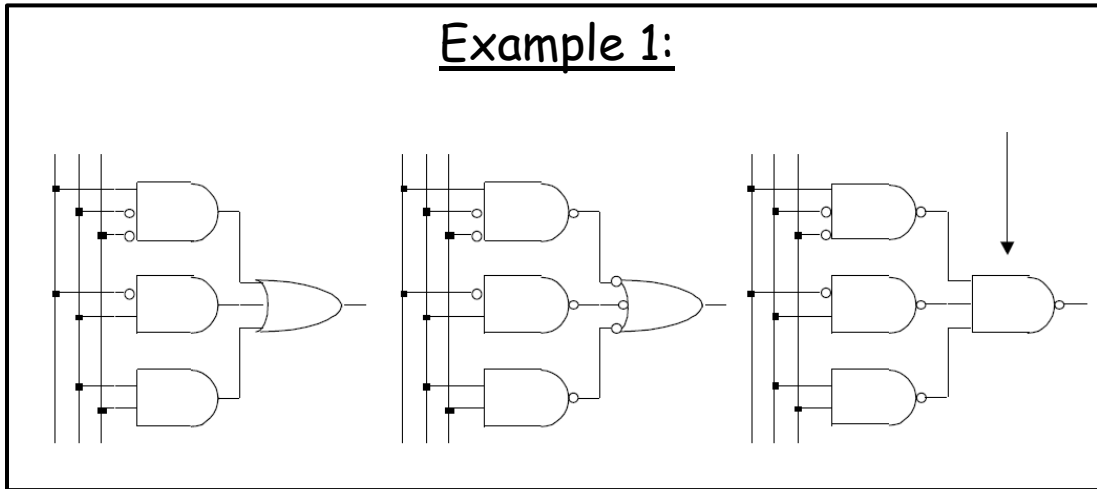
NAND gate implementation

$$\begin{aligned} Z &= (X.Y)' \leftarrow \text{NAND gate} \\ &= X' + Y' \quad (\text{using De Morgan's}) \end{aligned}$$

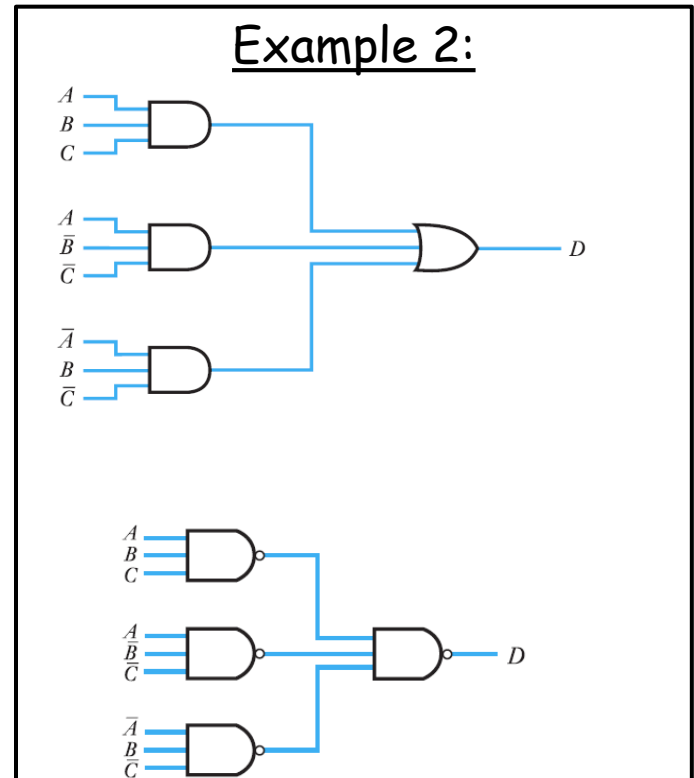


Implementing SOP using NAND gates only

Example 1:



Example 2:



LOGIC SIMPLIFICATION USING KARNAUGH MAP

- Mapping functions to Karnaugh map
- Karnaugh map rules
- Don't care condition
- Product of sums from Karnaugh map

Karnaugh Map Simplification

Each adjacent cell varies by only one bit.

2-variable map

		B	
		0	1
A	0	00 0	01 1
	1	10 2	11 3

4-variable map

		AB			
		00	01	11	10
CD	00	0	4	12	8
	01	1	5	13	9
	11	3	7	15	11
	10	2	6	14	10

3-variable map

		BC			
		00	01	11	10
A	0	A'B'C' 0	A'B'C 1	A'BC 3	A'BC' 2
	1	AB'C' 4	AB'C 5	ABC 7	ABC' 6



Truth tables and functions

Minterms/Maxterms
Each combination of the variables in a truth table in the form of a product term is called a minterm.

x	y	z	Minterm	Maxterm
0	0	0	$x'y'z' = m_0$	$x+yz = M_0$
0	0	1	$x'y'z = m_1$	$x+yz' = M_1$
0	1	0	$x'yz' = m_2$	$x+yz = M_2$
0	1	1	$x'yz = m_3$	$x+yz' = M_3$
1	0	0	$xy'z' = m_4$	$x'yz = M_4$
1	0	1	$xy'z = m_5$	$x'yz' = M_5$
1	1	0	$xyz' = m_6$	$x'yz = M_6$
1	1	1	$xyz = m_7$	$x'yz' = M_7$

Sum of Products
 $X = f(A,B,C) = \sum m(1,5,6)$
 $= \sum (1,5,6)$
 $= A'BC + ABC + ABC$

Product of Sums
 $X = f(A,B,C) = \prod M(0,2,3,4,7)$
 $= \prod (0,2,3,4,7)$
 $= (A+B+C)(A+B+C)(A+B+C)(A+B+C)(A+B+C)$

Note that $m_7 = M_7$

MA-38

Mapping functions to Karnaugh Maps

- $f_1(x, y, z) = \sum (2, 3, 5, 7)$
 $= x'y + xz$

X \ yz	yz			
	00	01	11	10
0	0	0	1	1
1	0	1	1	0

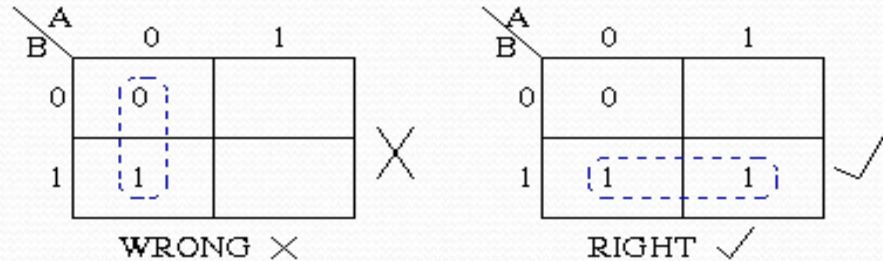
- $f_2(x, y, z) = \sum (0, 1, 2, 3, 6)$
 $= x' + yz'$

X \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	0	0	0	1

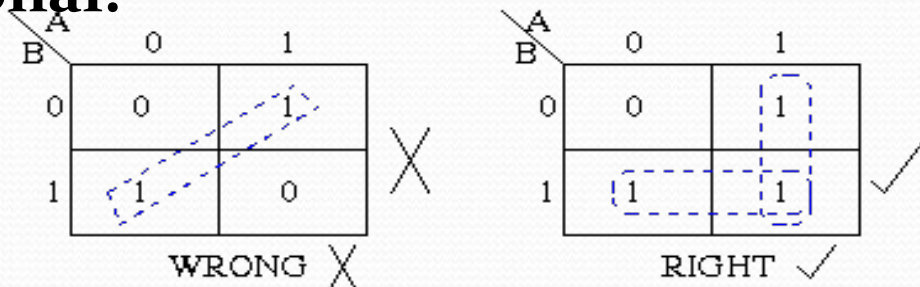
Karnaugh Maps - Rules of Simplification

The Karnaugh map uses the following rules for the simplification of expressions by grouping together adjacent cells containing *ones*.

1. Groups may not include any cell containing a zero.



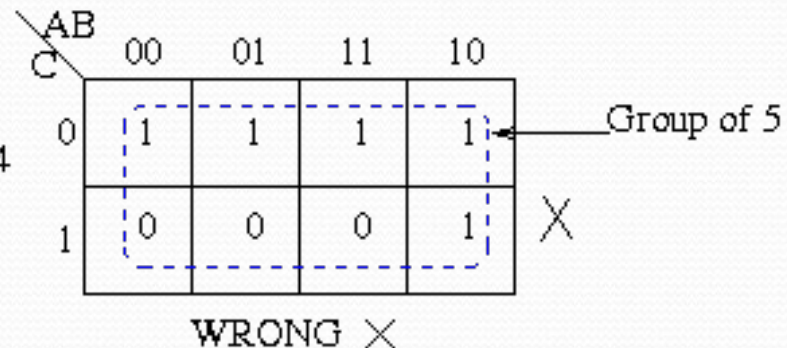
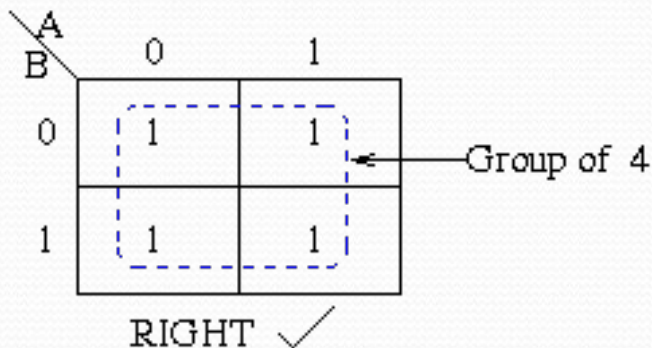
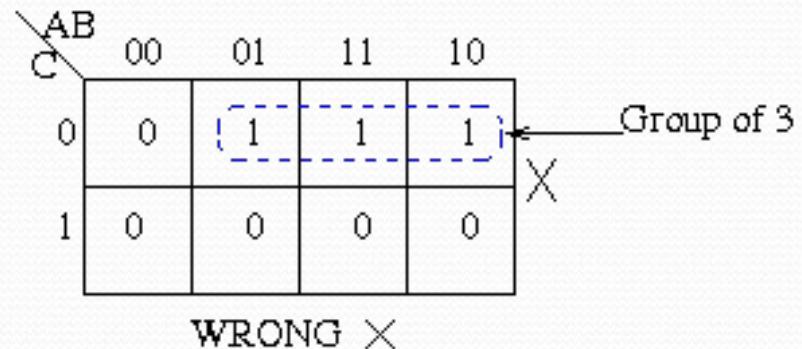
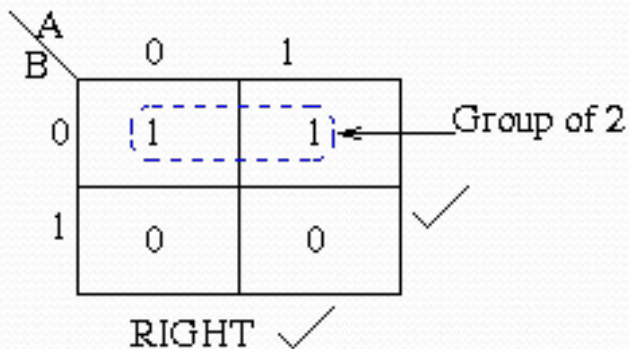
2. Groups may be horizontal or vertical, but not diagonal.



3. Groups must contain 1, 2, 4, 8, or in general 2^n cells.

That is if $n = 1$, a group will contain two 1's since $2^1 = 2$.

If $n = 2$, a group will contain four 1's since $2^2 = 4$.



4. Each group should be as large as possible.

$\overline{A}B$ C	00	01	11	10
0	1	1	1	1
1	0	0	1	1

RIGHT ✓

$\overline{A}B$ C	00	01	11	10
0	1	1	1	1
1	0	0	1	1

WRONG ✗

(Note that no Boolean laws broken,
but not sufficiently minimal)

5. Each cell containing a one must be in at least one group.

$\overline{A}B$ C	00	01	11	10
0	0	0	1	1
1	0	0	0	1

Group I
Group II

1 present in at least one group.

6.Groups may overlap.

AB					
C		00	01	11	10
0		1	1	1	1
1		0	0	1	1

Groups overlapping. ✓

RIGHT ✓

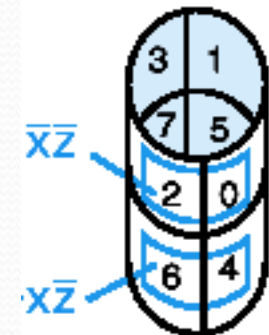
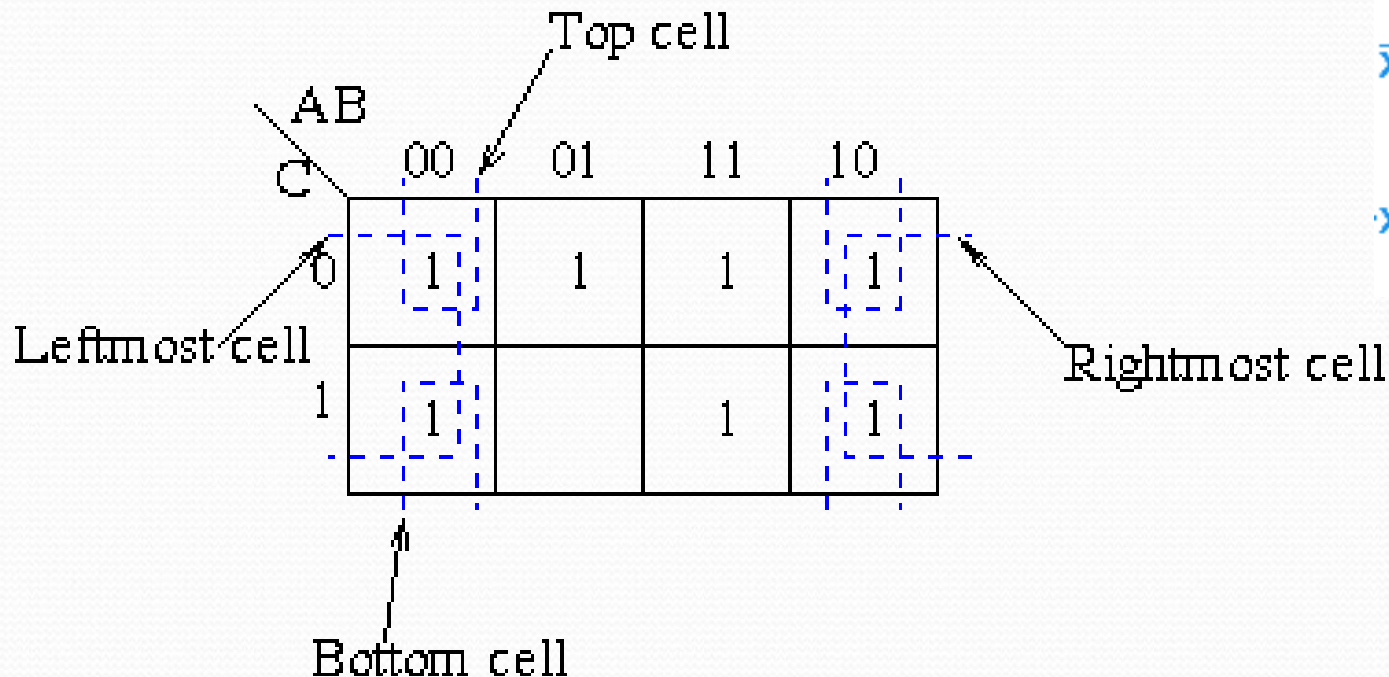
AB					
C		00	01	11	10
0		1	1	1	1
1		0	0	1	1

Groups not overlapping. ✗

WRONG ✗

7. Groups may wrap around the table.

The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.



8. There should be as few groups as possible, as long as this does not contradict any of the previous rules.

C \ AB	AB			
	00	01	11	10
0	1	1	1	1
1	0	0	1	1

RIGHT ✓

C \ AB	AB			
	00	01	11	10
0	1	1	1	1
1	0	0	1	1

WRONG ✗

Summary of Simplification Rules:

- No zeros allowed.
- No diagonals.
- Only power of 2 number of cells in each group.
- Groups should be as large as possible.
- Every one must be in at least one group.
- Overlapping allowed.
- Wrap around allowed.
- Fewest number of groups possible.

Map Simplification: Example 1

Simplify $f(x,y,z) = x'y'z + xy'z + xyz' + xyz$

Simplification using Boolean Algebra

$$\begin{aligned} f(x,y,z) &= x'y'z + xy'z + xyz' + xyz \\ &= y'z(x'+x) + xy(z+z') \\ &= y'z + xy \end{aligned}$$

Simplification using K-map

		yz			
		00	01	11	10
x	0		1		
	1		1	1	1

Diagram showing K-map cells with groupings:

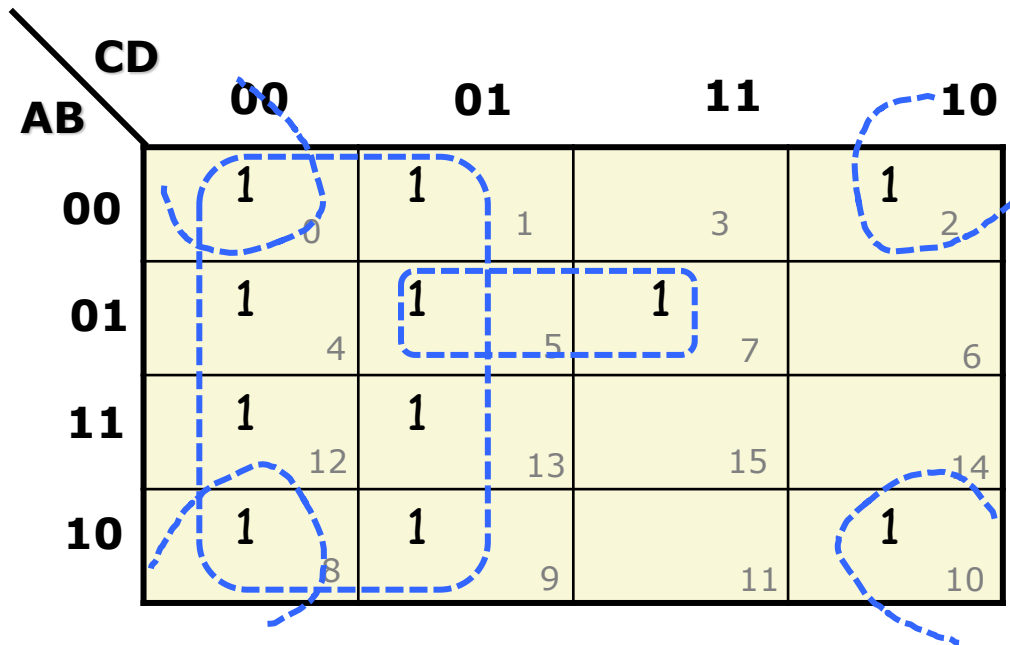
- Group 1 (Blue dashed box): Cells (0,1) and (1,1) containing 1 and 1.
- Group 2 (Green dashed box): Cells (1,1) and (1,3) containing 1 and 1.
- Group 3 (Red dashed box): Cells (1,2) and (1,3) containing 1 and 1.

$$f(x,y,z) = y'z + xy$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Map Simplification: Example 2

Simplify the following Boolean function
 $F(A,B,C,D) = \Sigma(0,1,2,4,5,7,8,9,10,12,13)$



$$F(A,B,C,D) = C' + B'D' + A'BD$$

Map Simplification: Example 3

$$F(X,Y,Z) = X'Y'Z + X'YZ + XY'Z + XYZ$$

yz		00	01	11	10
x	0		1	1	
	1		1	1	

0 1 3 2
4 5 7 6

$$F(x,y,z) = z$$

$$F(X,Y,Z) = X'Y'Z' + X'Y'Z + X'YZ + X'YZ' + XY'Z' + XYZ'$$

yz		00	01	11	10
x	0	1	1	1	1
	1	1			1

0 1 3 2
4 5 7 6

$$F(x,y,z) = x' + z'$$

Map Simplification: Example 4 (Make largest possible groups)

Example 1

CD \ AB	00	01	11	10
00	0	1	1 ₃	1 ₂
01	4	1 ₅	1 ₇	6
11	12	1 ₁₃	1 ₁₅	14
10	8	9	1 ₁₁	10

$$F = AB + A'BC'D + A'BCD + AB'C'D'$$

CD \ AB	00	01	11	10
00	0	1	1 ₃	1 ₂
01	4	1 ₅	1 ₇	6
11	12	1 ₁₃	1 ₁₅	14
10	8	9	1 ₁₁	10

$$F = AB + BD + AC'D'$$

Example 2

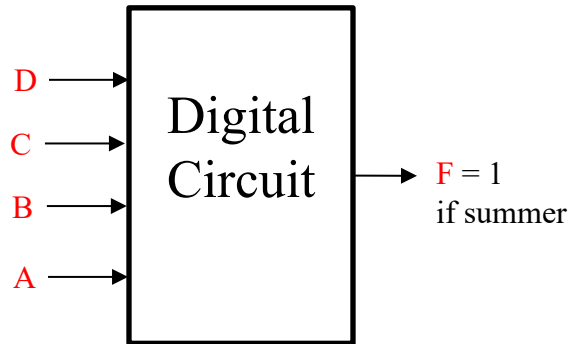
CD \ AB	00	01	11	10
00	0	1	1 ₃	1 ₂
01	1 ₄	1 ₅	7	1 ₆
11	1 ₁₂	13	15	1 ₁₄
10	1 ₈	9	11	10

$$F = AC'D' + A'C'D + A'B'C + AB'D$$

CD \ AB	00	01	11	10
00	0	1	1 ₃	1 ₂
01	1 ₄	1 ₅	7	1 ₆
11	1 ₁₂	13	15	1 ₁₄
10	1 ₈	9	11	10

$$F = AC'D' + A'C'D + A'B'C + B'D$$

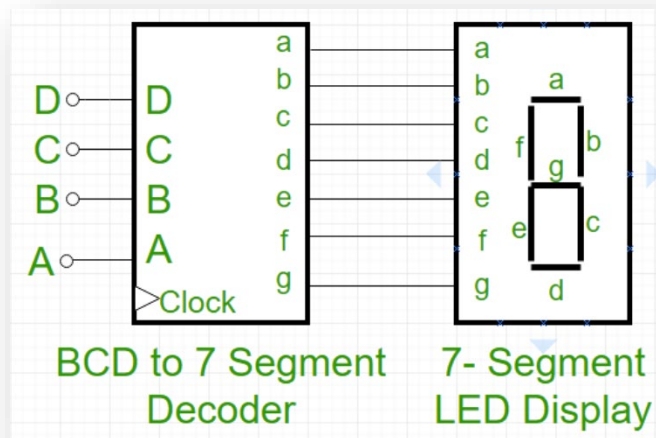
K-Map: Don't care condition example 1



DC \ BA	00	01	11	10
00	X ₀	0 ₁	0 ₃	0 ₂
01	0 ₄	0 ₅	1 ₇	1 ₆
11	0 ₁₂	X ₁₃	X ₁₅	X ₁₄
10	1 ₈	0 ₉	0 ₁₁	0 ₁₀

Inputs				Output	
D	C	B	A	Month	F
0	0	0	0		x
0	0	0	1	Jan	0
0	0	1	0	Feb	0
0	0	1	1	Mar	0
0	1	0	0	April	0
0	1	0	1	May	0
0	1	1	0	June	1
0	1	1	1	July	1
1	0	0	0	Aug	1
1	0	0	1	Sept	0
1	0	1	0	Oct	0
1	0	1	1	Nov	0
1	1	0	0	Dec	0
1	1	0	1		x
1	1	1	0		x
1	1	1	1		x

K-Map: Don't care condition example 2



Truth Table – For common cathode type BCD to seven segment decoder:

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

K-Map: Don't care condition

- You don't always need all 2^n input combinations in an n -variable function
 - If you can guarantee that certain input combinations never occur
 - If some outputs aren't used in the rest of the circuit
- We mark don't-care outputs in truth tables and K-maps with Xs.

x	y	z	$f(x,y,z)$
0	0	0	0
0	0	1	1
0	1	0	X
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	X
1	1	1	1

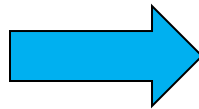
- Within a K-map, each X can be considered as either 0 or 1. You should pick the interpretation that allows for the most simplification

Map Simplification with Don't Care: Example 5

Given

$$f = \sum m(1,2,4,5,6,8,9) + d(10,11,14,15)$$

AB \ CD	00	01	11	10
00	0 	1 1	3 	2 1
01	1 4	1 5	7 	6 1
11	12 	13 	X 15	X 14
10	1 8	1 9	X 11	X 10



AB \ CD	00	01	11	10
00	0 	1 1	3 	2 1
01	1 4	1 5	7 	6 1
11	12 	13 	X 15	X 14
10	1 8	1 9	X 11	X 10

$$f = a'c'd + ab' + cd' + a'bc'$$

AB \ CD	00	01	11	10
00	0 	1 1	3 	2 1
01	1 4	1 5	7 	6 1
11	12 	13 	X 15	X 14
10	1 8	1 9	X 11	X 10

$$f = a'c'd + ab' + cd' + a'bd'$$

Map Simplification with Don't Care: More Examples

A \ BC	00	01	11	10
0				
1		1	x	x

$AB'C$

A \ BC	00	01	11	10
0				
1		1	x	x

AC

A \ BC	00	01	11	10
0		1	1	1
1	1	1	x	x

$A + B + C$

A \ BC	00	01	11	10
0			1	1
1	1	1	x	x

$A + B$

A \ BC	00	01	11	10
0			1	
1	1	1	x	x

$A + BC$

A \ BC	00	01	11	10
0				
1	1	1	x	x

A

A \ BC	00	01	11	10
0				
1		1	x	x

AC

Product of Sums

Boolean Functions of Three Variables

X	Y	Z	F	$\bar{F}$
0	0	0	1	0
0	0	1	0	1
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	1	0
1	1	0	0	1
1	1	1	1	0

Method 1

$$F' = X'Y'Z + X'YZ + XY'Z' + XYZ' \quad \leftarrow \text{Sum of Products}$$

Using De Morgan's theorem,

$$F = (F')' = (X'Y'Z + X'YZ + XY'Z' + XYZ')'$$

$$= (X'Y'Z)' (X'YZ)' (XY'Z')' (XYZ')'$$

$$= (X+Y+Z') (X+Y'+Z) (X'+Y+Z) (X'+Y'+Z) \quad \leftarrow \text{Product of Sums}$$

$$= (X+Z') (X'+Z) \quad \leftarrow \text{Using } (a+b)(a+b') = a$$

Method 2

		yz			
		00	01	11	10
x	0	1 0	0 1	0 3	1 2
	1	0 4	1 5	1 7	0 6

$$F = (X+Z') (X'+Z)$$

MA-56

De Morgan's Theorem

A key theorem in simplifying Boolean algebra expression is DeMorgan's Theorem. It states:

$$(a + b)' = a'b'$$

$$(ab)' = a' + b'$$

Example 1: Complement the expression $F = a(b + z(x + a'))$ and simplify.

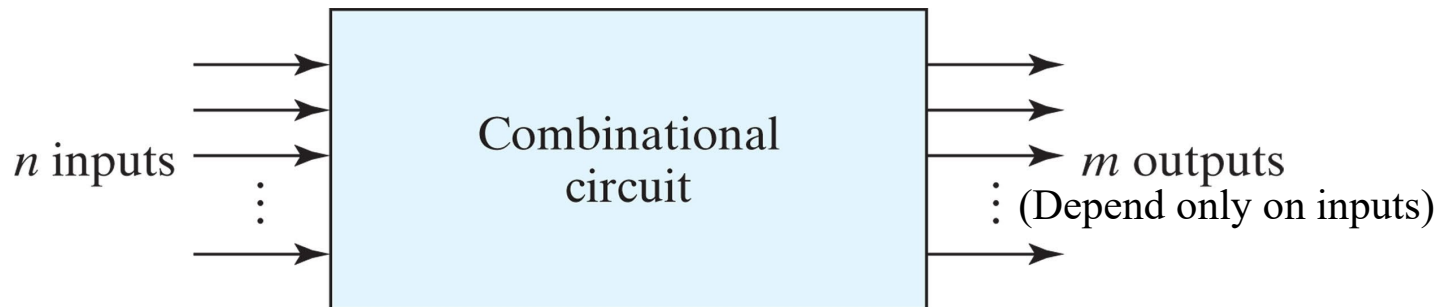
$$\begin{aligned} F' &= (a(b + z(x + a')))' \\ &= a' + (b + z(x + a'))' \\ &= a' + b' (z(x + a'))' \\ &= a' + b' (z' + (x + a')) \\ &= a' + b' (z' + x' + a) \end{aligned}$$

COMBINATIONAL CIRCUIT

- Half adder
- Full adder
- BCD to Excess-3 converter

Combinational Circuits

- Logic circuits for digital systems may be **combinational** or **sequential**.
- A combinational circuit consists of input variables, logic gates, and output variables.
- Combinational Circuit Design: Given a problem statement
 - Determine the number of inputs and outputs
 - Derive the truth table
 - Simplify the Boolean expression for each output
 - Produce the required circuit



Copyright ©2013 Pearson Education, publishing as Prentice Hall

Design a Half Adder (combinational circuit)

- A combinational circuit that performs the addition of two bits is called a **half adder**.

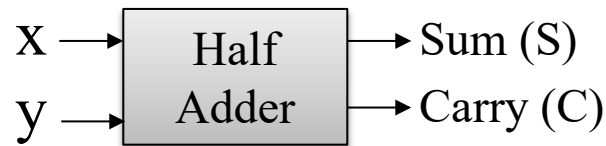


Table 4-3
Half Adder

<i>x</i>	<i>y</i>	<i>C</i>	<i>S</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Half Adder:
Truth Table

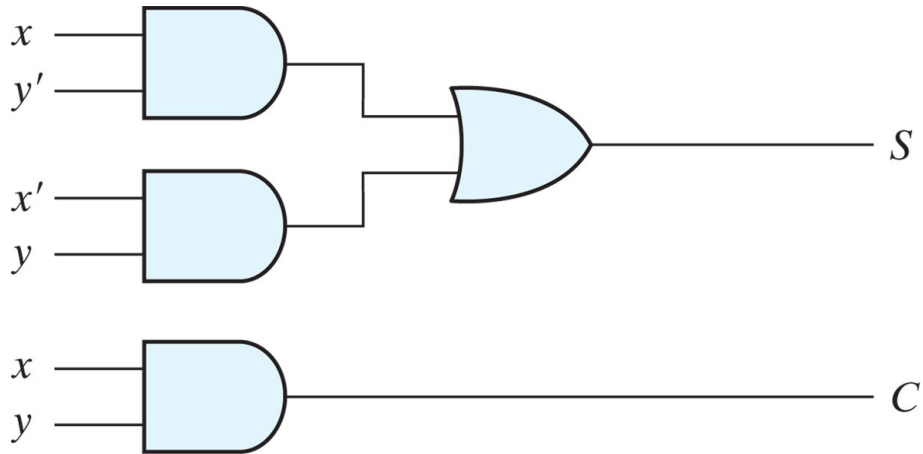
S: Sum
C: Carry

$$S = x'y + xy'$$

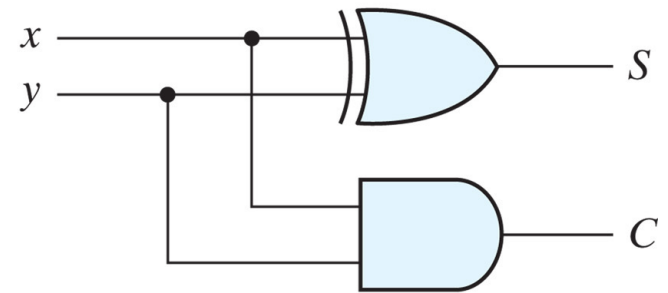
$$C = xy$$

Half Adder: Sum and Carry functions

Half Adder Implementation



(a) $S = xy' + x'y$
 $C = xy$



(b) $S = x \oplus y$
 $C = xy$

Copyright ©2013 Pearson Education, publishing as Prentice Hall

Design a Full Adder (combinational circuit)

- One that performs the addition of three bits (two significant bits and a previous carry) is a **full adder**.

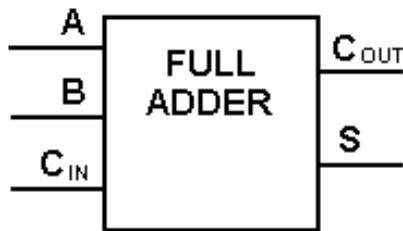
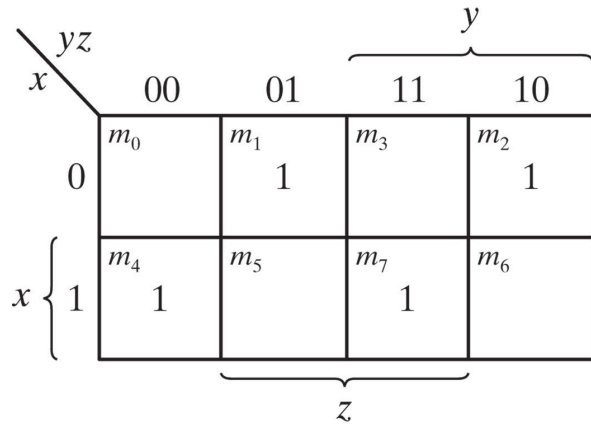


Table 4.4
Full Adder

<i>x</i>	<i>y</i>	<i>z</i>	<i>C</i>	<i>S</i>
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

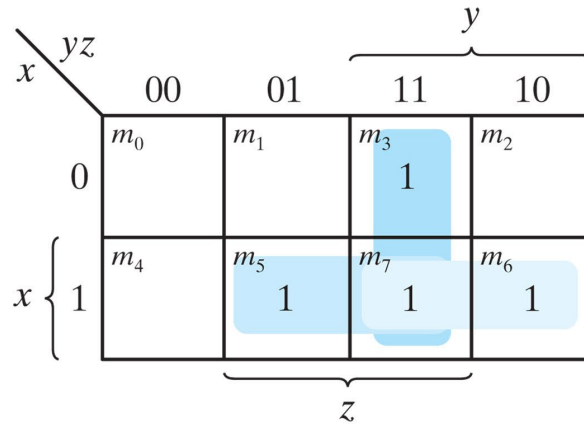
Copyright ©2012 Pearson Education, publishing as Prentice Hall

Full Adder: Sum and Carry functions



$$(a) S = x'y'z + x'yz' + xy'z' + xyz$$

Copyright ©2013 Pearson Education, publishing as Prentice Hall



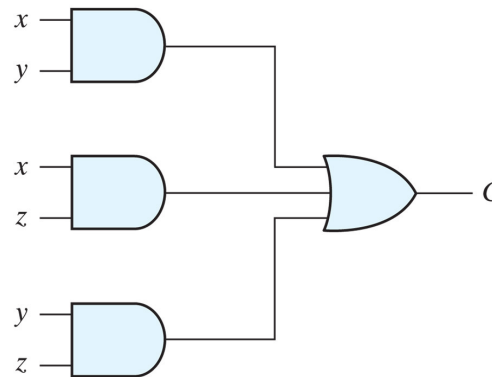
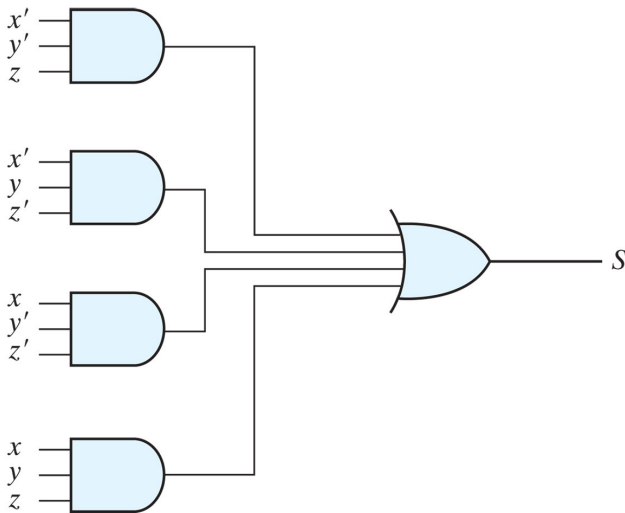
$$(b) C = xy + xz + yz$$

Table 4.4
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Copyright ©2013 Pearson Education, publishing as Prentice Hall

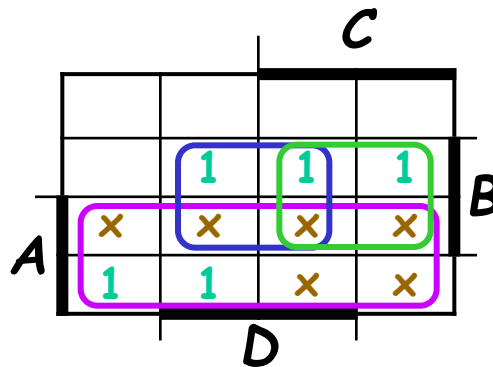
Full Adder: Implementation



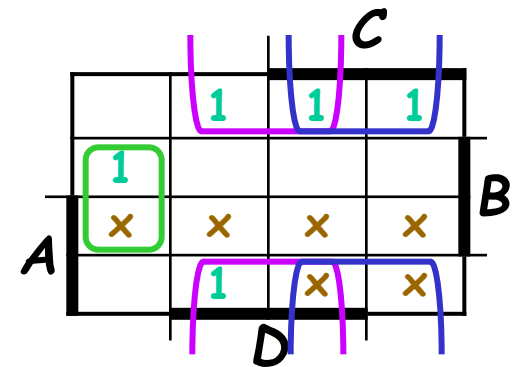
Copyright ©2013 Pearson Education, publishing as Prentice Hall

Design a BCD to Excess 3 converter

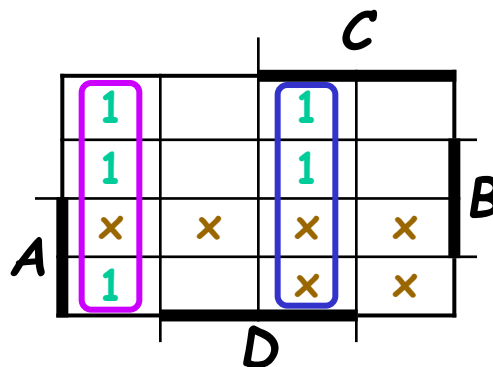
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x



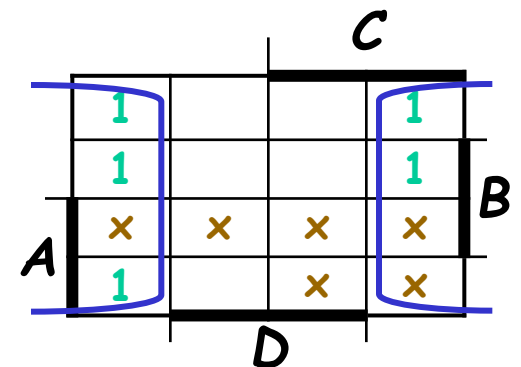
$$w = A + BC + BD$$



$$x = B'C + B'D + BC'D'$$



$$y = C'D' + CD$$



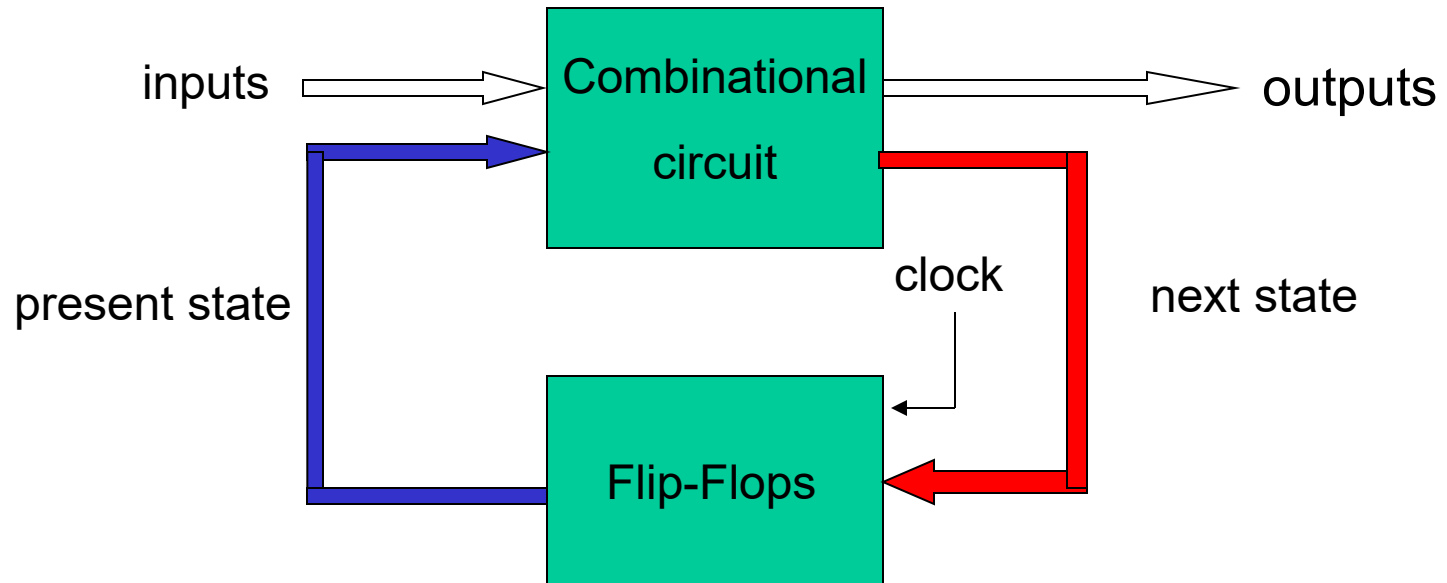
$$z = D'$$

SEQUENTIAL CIRCUIT

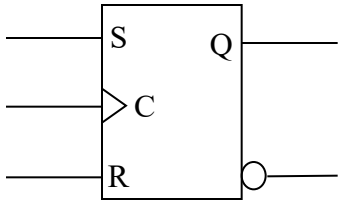
- Flip Flops
- Analysis of sequential circuits
- State table
- Flip Flop excitation tables
- Design of sequential circuit

Sequential Circuits

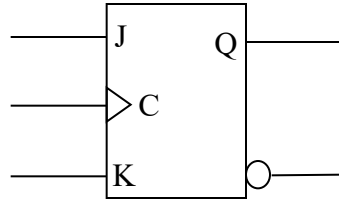
General Structure of Sequential Circuit



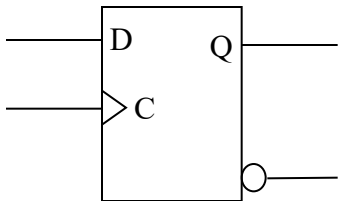
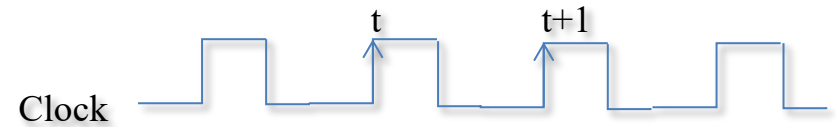
Flip Flops



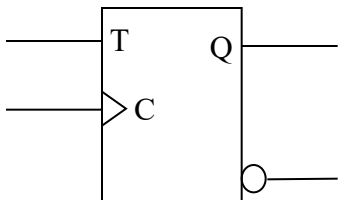
S	R	$Q(t+1)$	
0	0	$Q(t)$	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	?	Indeterminate



J	K	$Q(t+1)$	
0	0	$Q(t)$	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	$Q'(t)$	Complement



D	$Q(t+1)$	
0	0	Clear to 0
1	1	Set to 1

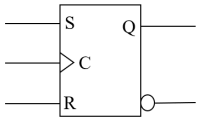


T	$Q(t+1)$	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

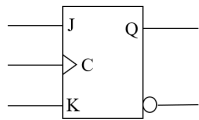


FF Excitation Tables

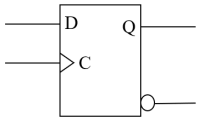
Flip Flops



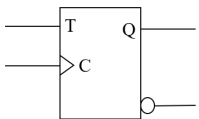
S	R	Q(t+1)	
0	0	Q(t)	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	?	Indeterminate



J	K	Q(t+1)	
0	0	Q(t)	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	Q'(t)	Complement



D	Q(t+1)	
0	0	Clear to 0
1	1	Set to 1



T	Q(t+1)	
0	Q(t)	No change
1	Q'(t)	Complement

SR Flip Flop

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

D Flip Flop

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

JK Flip Flop

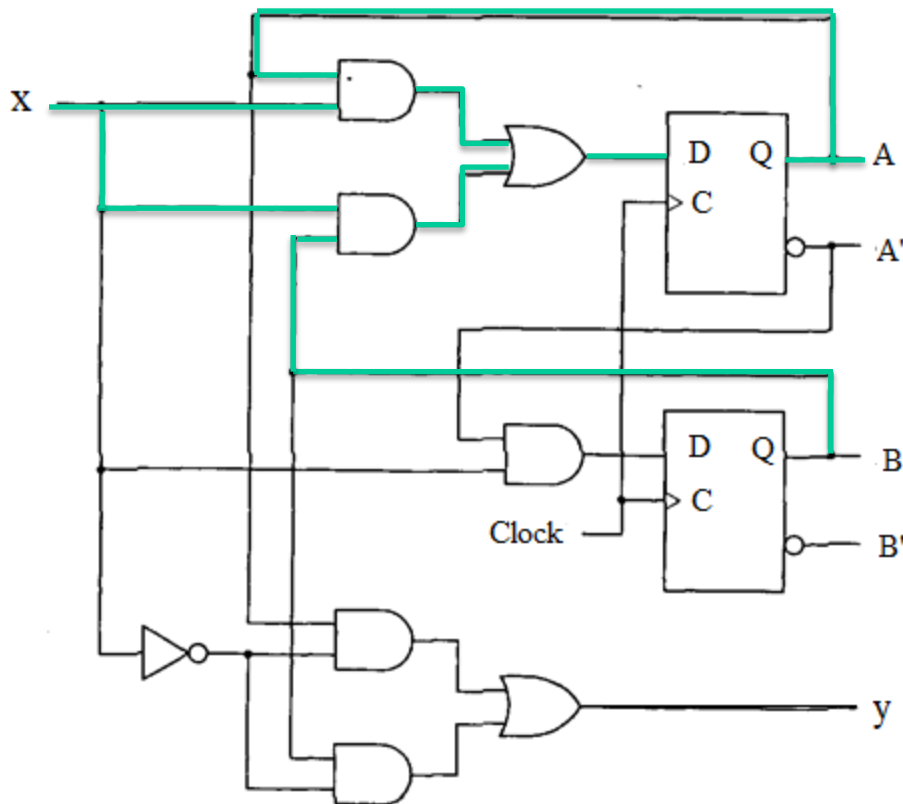
Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

T Flip Flop

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

Example: Analysis of a Sequential Circuit

Determine the state diagram of the following sequential circuit.



Flip-flop
input
equations:

$$D_A = Ax + Bx$$

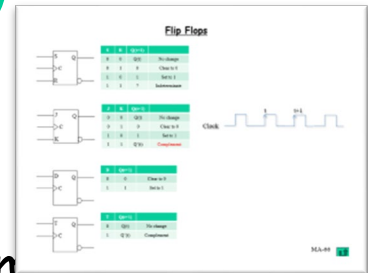
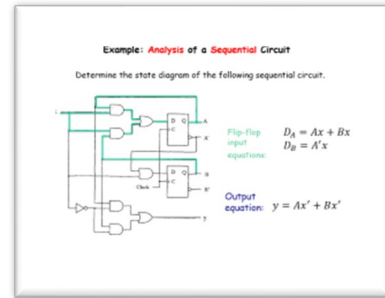
$$D_B = A'x$$

Output
equation: $y = Ax' + Bx'$

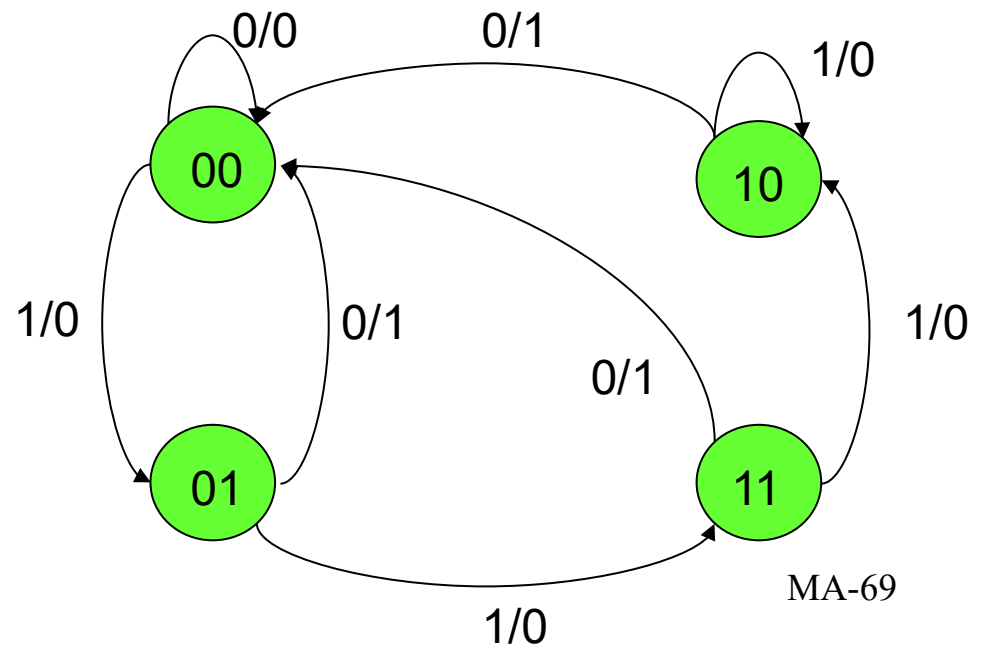
State Table

Present State		Input		Next State		Output
A	B	x		A	B	y
0	0	0		0	0	0
0	0	1		0	1	0
0	1	0		0	0	1
0	1	1		1	1	0
1	0	0		0	0	1
1	0	1		1	0	0
1	1	0		0	0	1
1	1	1		1	0	0

Present State
is used in
determining
Output



State Diagram



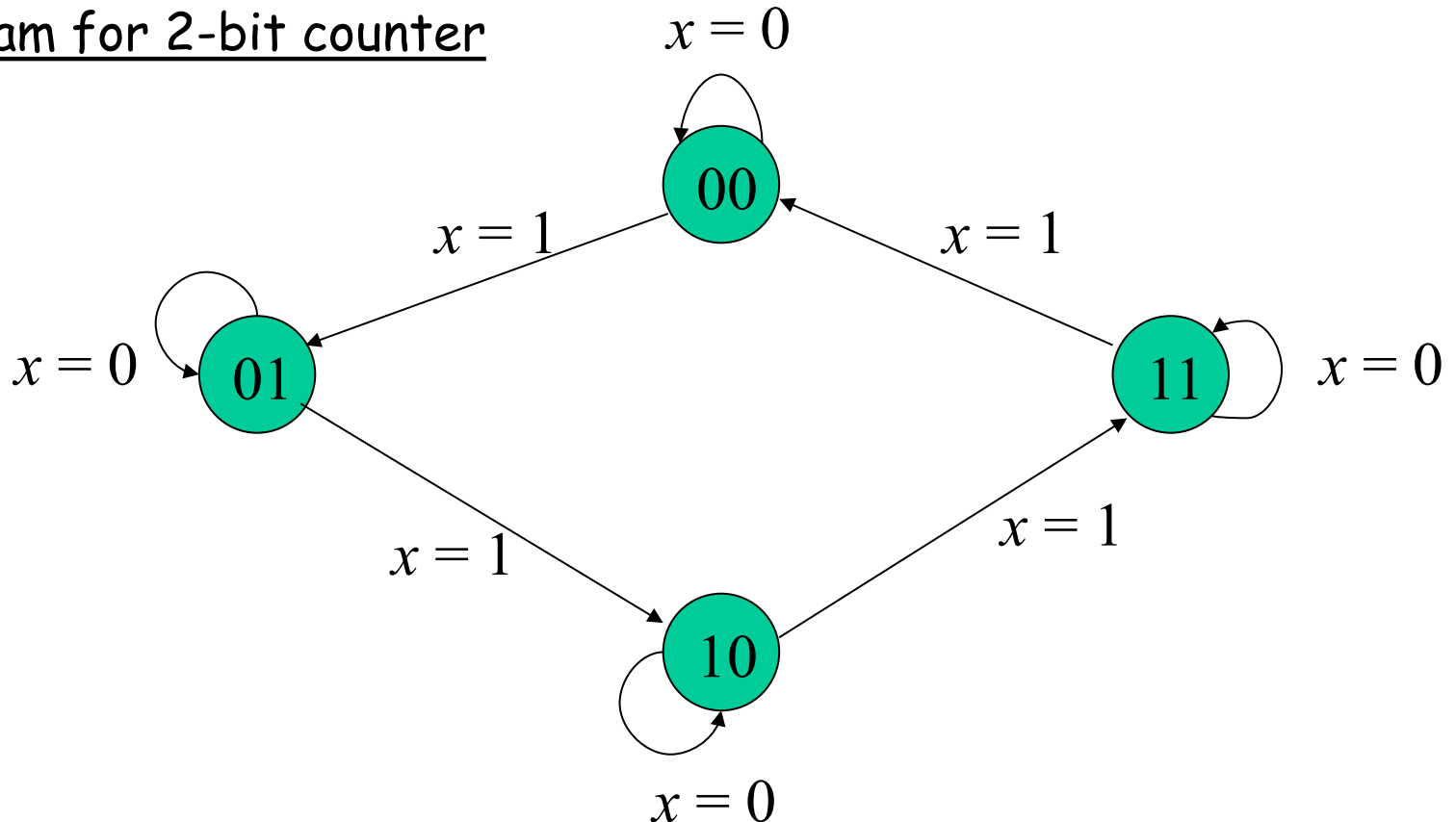
Flip-flop
input
equations:
 $D_A = Ax + Bx$
 $D_B = A'x$

Output
equation: $y = Ax' + Bx'$

Example: **Design** of a **Sequential** Circuit

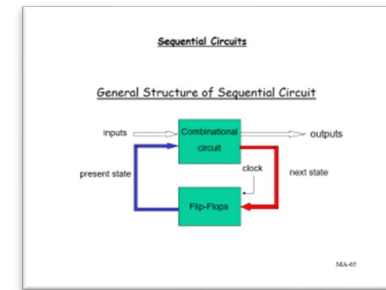
- Design a 2-bit counter
- Should count at every clock cycle if input (x) is one.

State Diagram for 2-bit counter



Excitation Table for 2-bit Counter

Designs for both D & JK flip-flops shown

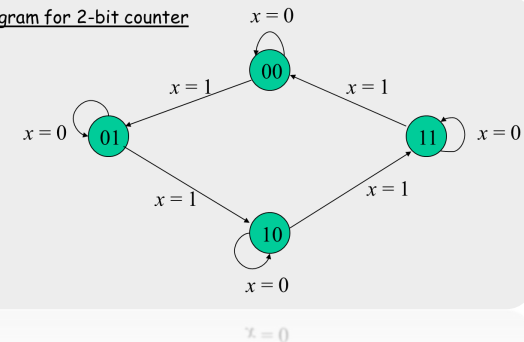


Excitation Table

Design 1

Design 2

State Diagram for 2-bit counter



x = 0

SR Flip Flop

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

D Flip Flop

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

JK Flip Flop

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

T Flip Flop

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

Present State		Input	Next State		D flip-flop inputs		JK flip-flop inputs			
A	B	x	A	B	D _A	D _B	J _A	K _A	J _B	K _B
0	0	0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	1	0	x	1	x
0	1	0	0	1	0	1	0	x	x	0
0	1	1	1	0	1	0	1	x	x	1
1	0	0	1	0	1	0	x	0	0	x
1	0	1	1	1	1	1	x	0	1	x
1	1	0	1	1	1	1	x	0	x	0
1	1	1	0	0	0	0	x	1	x	1

Design 1: Using D flip-flops

Present State		Input	D flip-flop inputs	
A	B	x	D _A	D _B
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

Excitation Table for 2-bit Counter

Designs for both D & JK flip-flops shown

Excitation Table

Present State	Next State	Next State	D flip-flop inputs	JK flip-flop inputs
A B	x	A B	D _A D _B	J _A K _A J _B K _B
0 0	0	0 0	0 0	0 1 0 1
0 0	1	0 1	0 1	0 1 0 1
0 1	0	0 0	0 0	0 1 1 0
0 1	1	0 1	0 1	0 1 1 0
1 0	0	1 0	1 0	1 0 0 1
1 0	1	1 1	1 1	1 0 0 1
1 1	0	1 0	1 0	1 0 1 0
1 1	1	1 1	1 1	1 0 1 0

Design 1

Design 2

JK flip-flop

D flip-flop

D_A

		B			
A \ Bx	A	0	0	1	0
		1	1	0	1

x

$$D_A = AB' + Ax' + A'Bx$$

D_B

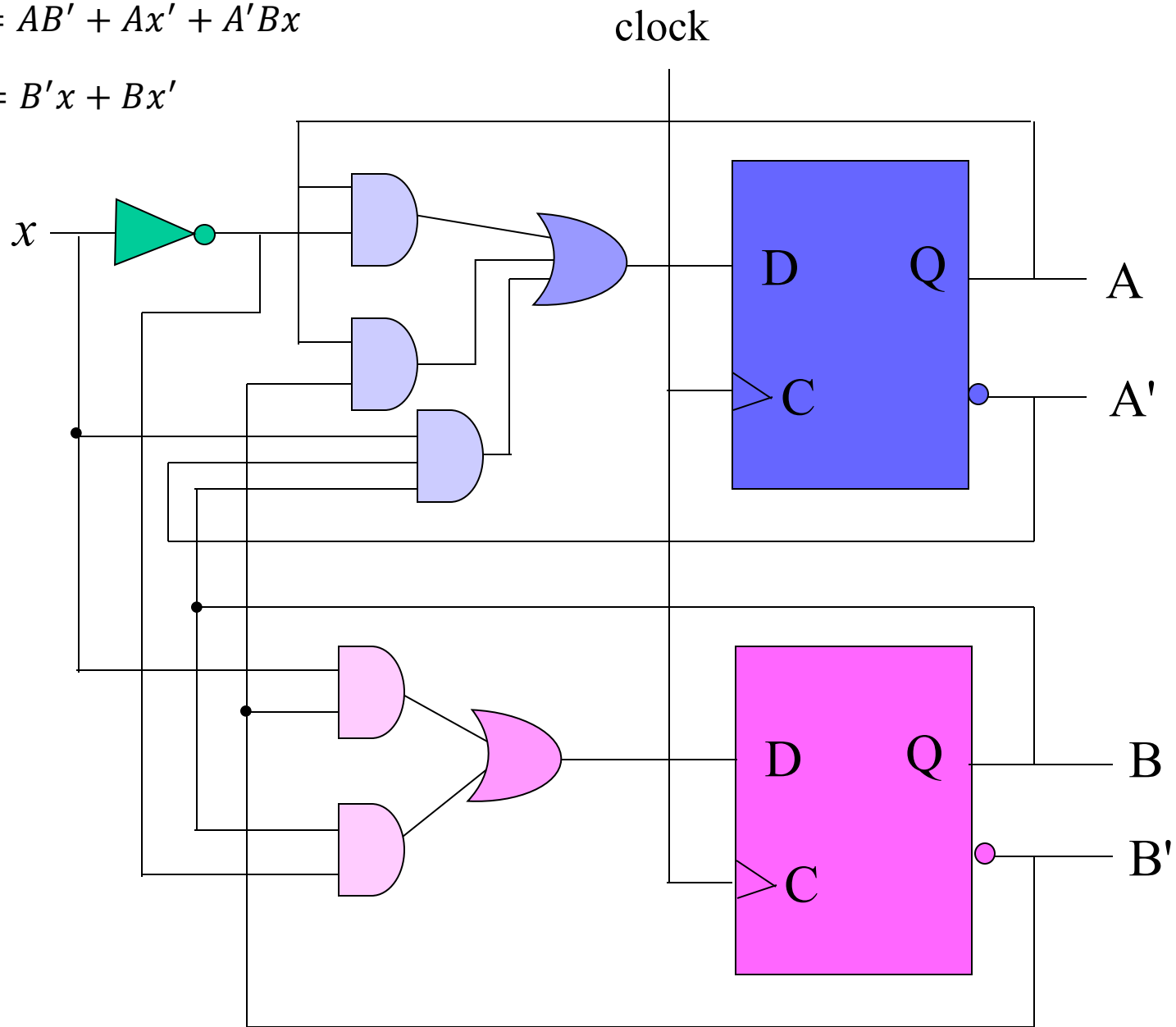
		B			
A \ Bx	A	0	1	0	1
		0	1	0	1

x

$$D_B = B'x + Bx'$$

$$D_A = AB' + Ax' + A'Bx$$

$$D_B = B'x + Bx'$$



Design 2: Using JK flip-flops

Present State		Input	JK flip-flop inputs (Combinational Circuit Outputs)			
A	B	x	J_A	K_A	J_B	K_B
0	0	0	0	x	0	x
0	0	1	0	x	1	x
0	1	0	0	x	x	0
0	1	1	1	x	x	1
1	0	0	x	0	0	x
1	0	1	x	0	1	x
1	1	0	x	0	x	0
1	1	1	x	1	x	1

Excitation Table for 2-bit Counter

Designs for both D & JK flip-flops shown

Excitation Table

Present State	Next State	Next State	D Flip-flop	JK Flip-flop
A	B	x	D _A	D _B
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

Design 1

Design 2

JK Flip-flop

D Flip-flop

J_A		B	
A	0	0	1
	x	x	x
		x	

$$J_A = Bx$$

K_A		B	
A	x	x	x
	0	0	1
		x	

$$K_A = Bx$$

			<u>JK flip-flop inputs</u> <u>(Combinational</u> <u>Circuit Outputs)</u>			
<u>Present State</u>		<u>Input</u>	<u>J_A</u>	<u>K_A</u>	<u>J_B</u>	<u>K_B</u>
A	B	x				
0	0	0	0	x	0	x
0	0	1	0	x	1	x
0	1	0	0	x	x	0
0	1	1	1	x	x	1
1	0	0	x	0	0	x
1	0	1	x	0	1	x
1	1	0	x	0	x	0
1	1	1	x	1	x	1

J_B

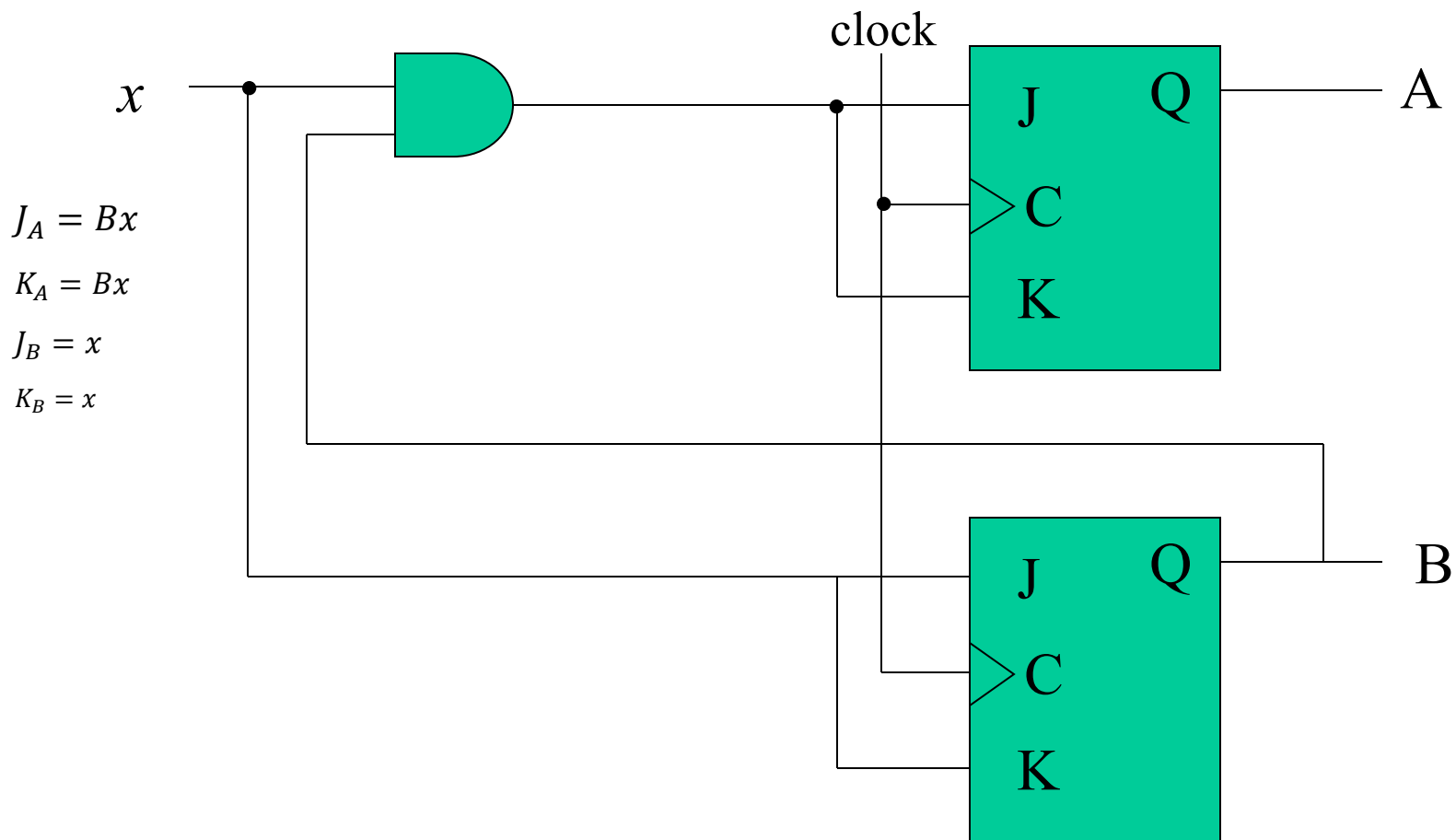
		B	
		{	
A {	0	1	x
	0	1	x
		{	
		x	

$$J_B = x$$

K_B

		B	
		{	
A {	x	x	1
	x	x	1
		{	
		x	

$$K_B = x \quad \text{MA-75}$$



Q: When the J & K inputs are connected together (like above) how does a JK flip-flop behave?

A: Like a T flip-flop.